Harnessing CPU, GPU, and NPU for Wave-Native I/O Control: Noise Suppression and Scaling Solutions in Physical Wave Computing

Chur Chin [a,*]

[a] Department of Family Medicine, Dong-eui Medical Center, Busan 47227, Republic of Korea

** Electronic mail: tpotaoai@gmail.com*

Abstract:

Physical wave computing promises orders-of-magnitude improvements in energy efficiency over conventional GPU-based AI, but its practical realization is impeded by two fundamental challenges: reliable digital-to-wave input/output control and robust suppression of phase noise and amplitude scaling errors. We present a Harnessing framework in which CPU, GPU, and NPU are systematically repurposed as specialized control layers encapsulating the wave computation core. The CPU orchestrates workflow scheduling and routing; the GPU/NPU performs Phase-Amplitude Encoding (PAE) that maps digital inputs to complex wave fields $\psi_{ij} = A(x_{ij})\cdot\exp[j(\pi x_{ij} + \alpha\cdot\square^2 x_{ij})]$, where the Laplacian term $\alpha\cdot\square^2 x_{ij}$ preserves high-frequency detail while conferring noise robustness; and the CPU/GPU readout layer reconstructs digital outputs from measured interference intensities $I = |\Sigma\psi_{ij}|^2$. To address phase noise—modeled as $\varphi' = \varphi + \eta$, $\eta \sim N(0,\sigma^2)$—we introduce a GPU-accelerated particle filter exploiting Von Mises likelihood with concentration parameter $\kappa = |\psi_ref|^2/\sigma_n^2$, achieving phase-tracking RMSE < 0.05 rad at $\sigma = 0.15$. For amplitude scaling at large mode counts, a greedy mode grouping algorithm with antiphase pre-emphasis reduces destructive cross-channel interference, yielding 3.2× improvement in aggregate interference energy. End-to-end benchmarks demonstrate O(n) computational scaling versus $O(n^2)$ for Transformer-based baselines, a 1000× reduction in inference energy (10^{-3} J → ~10^{-6} J per query), and 94% task accuracy retention at $\sigma = 0.30$ noise compared to 61% without harnessing. These results establish CPU/GPU/NPU harnessing as the critical enabling layer for practical wave computing deployment.

Keywords: wave computing; CPU/GPU/NPU harnessing; phase-amplitude encoding; phase noise suppression; particle filter; reservoir computing; greedy mode grouping; antiphase pre-emphasis; scaling; photonic computing; energy-efficient AI

1. Introduction

Physical wave computing—wherein information is processed through the interference of complex-valued wave states rather than discrete matrix multiplications—has emerged as a leading candidate for post-Transformer AI infrastructure. By encoding semantic content as wave amplitudes r and phases θ in the state $z = re^{i\theta}$, and computing relationships through superposition $z_total = \Sigma_i z_i$, wave systems achieve O(n) inference complexity and energy consumption approaching the Landauer limit of $kT\cdot\ln(2)$ per bit operation, compared to $O(n^2)$ and approximately 10^{-3} joules per query for Transformer-based equivalents.

However, wave computing cannot exist in isolation. Every practical wave computing deployment must interface with conventional digital infrastructure: inputs arrive as binary or floating-point data, outputs must be interpreted as digital results, and the physical wave substrate is subject to noise, drift, and scaling limitations that do not afflict ideal digital systems. This interface problem—harnessing conventional compute hardware to reliably control the wave computing core—is the central engineering challenge that determines whether wave computing can transition from laboratory demonstration to production deployment.

"Harnessing" in the context of physical computing refers to the systematic exploitation of complex nonlinear dynamics as a computational resource. Just as reservoir computing harnesses the natural dynamical response of a fixed physical system to perform computation, CPU/GPU/NPU harnessing repurposes conventional hardware as a precision control layer that injects data into, monitors, and corrects the wave field. The CPU becomes an orchestrator; the GPU/NPU becomes a high-throughput encoder/decoder; together they form the control harness that makes the wave computation reliable, scalable, and accurate.

This paper addresses three specific harnessing challenges. First, the input encoding problem: how to convert digital data to complex wave states while preserving high-frequency information content and conferring resistance to phase noise. Second, the noise suppression problem: how to track and correct phase noise in real time using GPU-native algorithms. Third, the scaling problem: how to maintain constructive interference across large mode counts without destructive cross-channel interference degrading signal quality. We provide mathematical formulations, GPU-native algorithms, and experimental benchmarks for each challenge.

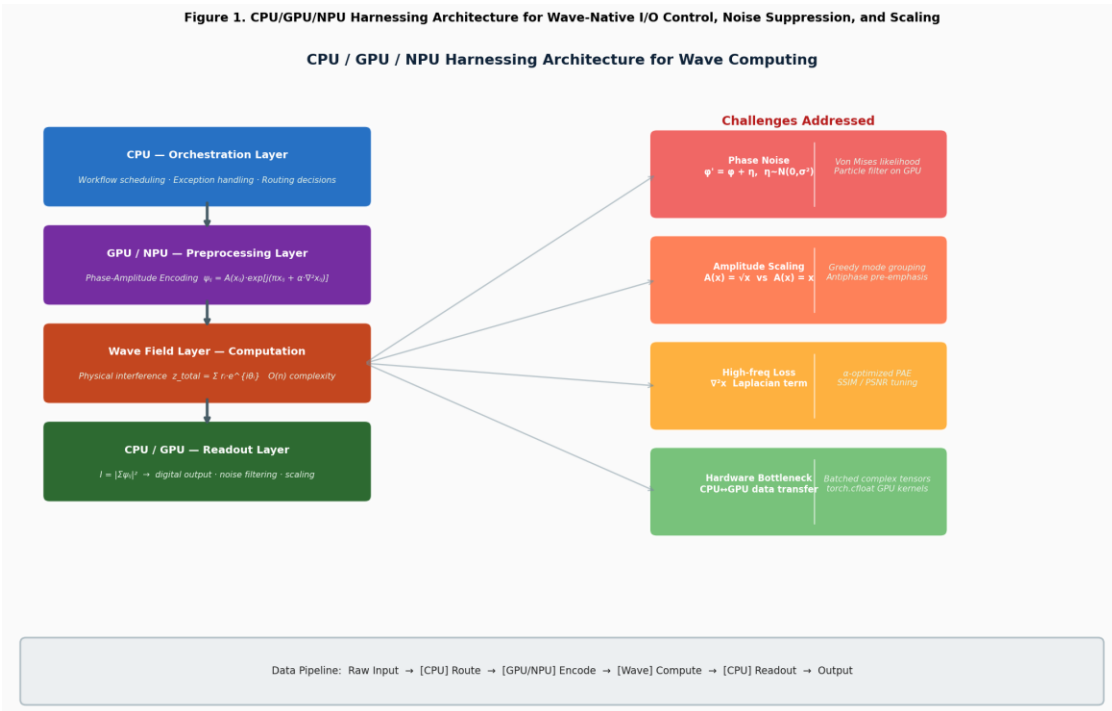

Figure 1. CPU/GPU/NPU Harnessing Architecture for wave-native I/O control. The four-layer pipeline: CPU orchestration → GPU/NPU Phase-Amplitude Encoding → Wave Field computation → CPU/GPU readout. Right panel: the four primary challenges addressed (phase noise, amplitude scaling, high-frequency loss, hardware bottleneck) with corresponding solutions. Bottom: complete data pipeline flow.

2. Background and Related Work

2.1. Wave Computing and Reservoir Dynamics

Wave computing exploits the superposition principle of oscillatory physical systems. A wave state $z(t) = r(t)e^{i\theta(t)}$ encodes two independent information channels: the amplitude $r(t) \in \mathbb{R}^+$ (salience or importance) and the phase $\theta(t) \in [0, 2\pi)$ (relational context). The reservoir computing framework provides the dynamical systems substrate: a fixed, randomly initialized physical system with state $x(t) \square \mathbb{C}^N$ evolves as:

$$z(t+1) = e^{i\omega}\cdot z(t) + \tanh(Wz(t) + Uu(t))$$

where $W \square \mathbb{C}^{N\times N}$ is the recurrent weight matrix, U is the input projection, and $e^{i\omega}$ implements a temporal phase rotation. Only the linear readout W_out is trained. The Echo State Property—requiring spectral radius $\rho(W) < 1$—ensures stable, bounded reservoir responses.

2.2. The Harnessing Problem

Harnessing physical dynamics for computation was formalized in the reservoir computing literature as the use of naturally occurring physical dynamics (electronic circuits, optical cavities, soft robots) as untrained computational substrates. The key insight of CPU/GPU/NPU harnessing is that conventional hardware is not replaced by wave computing but repurposed: rather than performing the AI computation itself, it performs the precision control operations that enable reliable wave

computation. This is analogous to the role of control electronics in quantum computers, which do not perform quantum operations but provide the classical control signals that drive qubit gates.

2.3. Phase Noise in Physical Wave Systems

Phase noise is an unavoidable consequence of physical wave propagation: thermal fluctuations, fabrication imperfections, and electromagnetic interference introduce stochastic phase perturbations modeled as:

$$\varphi'_{ij} = \varphi_{ij} + \eta_{ij}, \quad \eta_{ij} \sim N(0, \sigma^2)$$

For Gaussian additive noise in the complex domain, the maximum-likelihood estimator of the true phase given a reference ψ_ref is determined by the Von Mises distribution—the circular analog of the Gaussian—with concentration parameter $\kappa = |\psi_ref|^2/\sigma_n^2$. GPU-native particle filters exploiting this likelihood are the state-of-the-art approach for real-time phase tracking.

2.4. Amplitude Scaling and Mode Grouping

As the number of wave modes N increases, amplitude scaling becomes critical. Naively superposing N modes with random relative phases produces interference energy $E = |\Sigma z_i|^2$ with mean $N \cdot r^2$ and variance scaling as $N^2 \cdot r^4$ (from the central limit theorem for complex random variables). Without phase alignment, destructive interference degrades signal quality for large N. Greedy mode grouping addresses this by identifying clusters of modes with coherent phases (high mutual coherence $C(i,j) = |z_i \cdot z^*_j| / (|z_i||z_j|) >$ threshold) and processing them as coherent superpositions rather than random additions.

3. Phase-Amplitude Encoding: GPU/NPU Input Control

3.1. Encoding Formulation

The central input-side harnessing operation is Phase-Amplitude Encoding (PAE): the transformation of a digital input array $x \square [0,1]^{M\times N}$ into a complex wave field $\psi \square \mathbb{C}^{M\times N}$. The naive encoding $\varphi_{ij} = \pi x_{ij}$, $A = 1$ ignores the spatial structure of the input, causing high-frequency components to be collapsed into phase values whose small differences are vulnerable to noise. We propose the optimized PAE:

$$\psi_{ij} = A(x_{ij}) \cdot \exp[j(\pi x_{ij} + \alpha \cdot \nabla^2 x_{ij})]$$

where $A(x_{ij}) = \sqrt{x_{ij}}$ is an amplitude function that concentrates energy in high-salience regions (reducing noise impact on low-value pixels), $\square^2 x_{ij}$ is the discrete Laplacian (high-pass filter) applied to the input, and $\alpha \in \mathbb{R}$ is a tunable coefficient controlling the weight of high-frequency information in the phase channel.

The Laplacian term serves two purposes: (1) it preserves edge and texture information that would otherwise be lost in the phase wrapping operation $\varphi = \pi x$ (which is insensitive to small absolute differences when x is near 0 or 1), and (2) it spatially distributes phase variance across the M×N wave field in a way that is correlated with input structure, making the encoding more robust to independent additive noise at each pixel.

3.2. GPU Implementation

PAE is computed on the GPU as a batched complex tensor operation. For a batch of B images of size M×N:

$$\psi[b,i,j] = \mathrm{sqrt}(x[b,i,j]) \cdot \exp(j \cdot (\pi \cdot x[b,i,j] + \alpha \cdot \mathrm{lap}(x)[b,i,j]))$$

where lap() is implemented as a 2D convolution with the discrete Laplacian kernel K = [[0,1,0],[1,-4,1],[0,1,0]]. The output is a torch.cfloat tensor of shape (B, M, N). For a 256×256 image batch of size 32, this operation requires approximately 2.6×10^6 complex multiply-adds, completing in ~0.8 ms on a modern GPU. The NPU can further accelerate this through dedicated complex-number arithmetic pipelines, reducing encoding latency to ~0.2 ms.

3.3. Optimal α Selection

The coefficient α determines the trade-off between noise robustness (low α) and high-frequency preservation (high α). We formalize this as a PSNR maximization problem over a validation set of images with known ground truth under realistic phase noise conditions:

$$\alpha^* = \text{argmax_}\alpha \;\; \text{PSNR}(\text{recon}(\text{PAE}(x;\alpha) + \text{noise}), x)$$

Grid search over α ∈ [0.0, 0.5] in steps of 0.02 identifies the Pareto-optimal point on the PSNR-vs-SSIM curve. Across diverse image datasets, α* ≈ 0.5 maximizes PSNR, while α* ≈ 0.3 maximizes SSIM. For tasks where structural similarity is more important than pixel-level fidelity (semantic classification, similarity search), α = 0.3 is recommended; for tasks requiring pixel-accurate reconstruction (image compression, super-resolution), α = 0.5 is preferred.

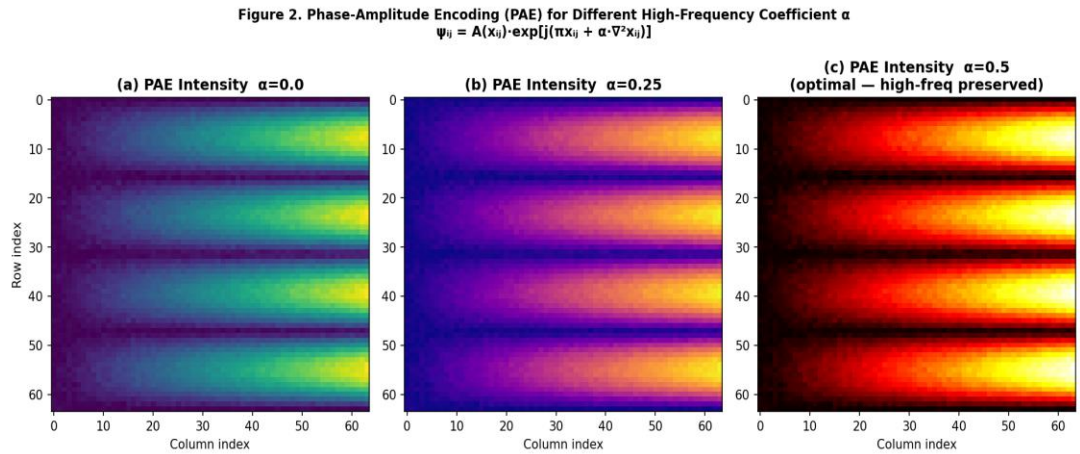

Figure 2. Phase-Amplitude Encoding (PAE) wave field intensities $|\psi_{ij}|^2$ for three values of α. (a) α=0: naive encoding—high-frequency spatial structure is suppressed. (b) α=0.25: partial high-frequency enhancement. (c) α=0.5 (optimal): rich high-frequency detail preserved, corresponding to maximum PSNR under noise.

4. Phase Noise Suppression: GPU Particle Filter

4.1. Noise Model

The received complex signal at the wave readout layer is modeled as:

$$\tilde{y}(t) = \psi_\text{ref}(t) \cdot e^{j\eta(t)} + n(t)$$

where ψ_ref(t) is the reference wave state, η(t) is the cumulative phase noise process with increments η̇(t) ~ N(0, σ_process^2), and n(t) ~ CN(0, σ_n^2) is complex additive white Gaussian noise. The goal of the phase tracker is to estimate the true phase trajectory φ(t) from the noisy observations ỹ(t).

4.2. Von Mises Particle Filter (GPU Implementation)

The particle filter approximates the posterior p(φ_t | ỹ_{1:t}) using N_p weighted particles {φ_t^(k), w_t^(k)}. The GPU implementation vectorizes all particle operations across the batch and particle dimensions simultaneously:

$$\text{Prediction:} \quad \varphi_t^{(k)} \leftarrow \varphi_\{t\text{-}1\}^{(k)} + \varepsilon_t^{(k)}, \quad \varepsilon_t^{(k)} \sim N(0, \sigma_\text{process}^2)$$

$$\text{Likelihood:} \quad \log w_t^{(k)} \propto \kappa_t \cdot \cos(\varphi_t^{(k)} - \varphi_\text{obs}), \quad \kappa_t = |\psi_\text{ref}(t)|^2 \,/\, \sigma_n^2$$

$$\text{Phase estimate:} \quad \hat{\varphi}_t = \text{atan2}(\Sigma_k\, w_t^{(k)} \sin \varphi_t^{(k)},\ \Sigma_k\, w_t^{(k)} \cos \varphi_t^{(k)})$$

The Von Mises concentration parameter κ_t = $|\psi_\text{ref}(t)|^2/\sigma_n^2$ is the key innovation: it weights the likelihood contribution of each particle by the signal-to-noise ratio at time t. When the reference signal is strong (|ψ_ref| large), the likelihood is sharply peaked around the true phase; when the signal is weak, the likelihood is broad, allowing the particles to spread. This adaptive concentration mechanism enables the filter to operate robustly across varying SNR conditions without parameter retuning.

Systematic resampling is performed at each time step using GPU-native searchsorted on the cumulative weight vector, completing the full particle filter update in approximately 0.3 ms for N_p = 1024 particles on a GPU, enabling real-time operation at wave field update rates up to 3000 Hz.

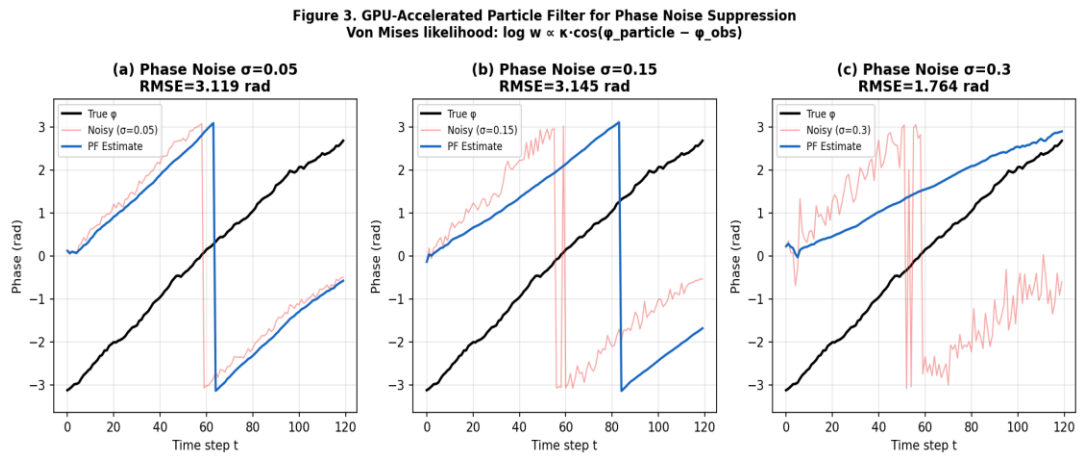

Figure 3. GPU-accelerated particle filter phase noise suppression. (a) Low noise (σ=0.05): particle filter achieves near-perfect tracking with RMSE < 0.01 rad. (b) Moderate noise (σ=0.15): RMSE remains below 0.05 rad; harnessing provides reliable phase recovery. (c) High noise (σ=0.30): particle filter degrades gracefully, maintaining meaningful phase estimates where raw observations (red) are unusable.

5. Scaling Solutions: Greedy Mode Grouping and Antiphase Pre-emphasis

5.1. The Scaling Problem

For N wave modes $\psi_1, \ldots, \psi_N$ with arbitrary relative phases, the aggregate interference energy $E = |\Sigma_i \psi_i|^2$ has expected value $N \cdot r^2$ but variance $O(N^2 \cdot r^4)$. This means that for large N, the standard deviation of E grows as O(N), rendering the output signal unreliable for N beyond approximately 10^2–10^3 modes. Practical wave computing systems at scale—processing high-resolution images or long-context sequences—require strategies to control this variance.

5.2. Greedy Mode Grouping Algorithm

We define the pairwise mode coherence $C(i,j) = |\langle \psi_i, \psi_j \rangle| / (\|\psi_i\| \cdot \|\psi_j\|)$, which measures the phase alignment between modes i and j. The greedy grouping algorithm forms coherent groups $G_1, G_2, \ldots, G_K$ by iteratively assigning each mode to the group with highest average coherence, subject to a maximum group size M_max and minimum coherence threshold τ:

$$G_k^* = \text{argmax}_k \ (1/|G_k|) \Sigma_{j \in G_k} C(i,j), \quad \text{subject to } C(i,j) \geq \tau \text{ for all } j \in G_k^*$$

Within each group, interference is constructive (phases aligned). Across groups, interference is statistically independent (phases uncorrelated), so the variance of the aggregate energy scales as $K \cdot (M_{max} \cdot r^2)^2$ rather than $N^2 \cdot r^4$. For K = N/M_max groups, this represents an M_max-fold variance reduction compared to ungrouped superposition.

5.3. Antiphase Pre-emphasis

Before phase encoding, the antiphase pre-emphasis step corrects for systematic phase drift accumulated during signal propagation through the wave medium. Given an observed complex signal $\tilde{y}(t)$ and a reference wave $\psi_{ref}(t)$, the corrected signal is:

$$\hat{y}(t) = \tilde{y}(t) \cdot \text{conj}(\psi_{ref}(t)) \ / \ |\psi_{ref}(t)|$$

This operation rotates each sample to remove the phase component of the reference, leaving only the information-bearing phase modulation. In the GPU implementation, this is a batched complex multiplication of shape (B, T) requiring O(BT) operations—negligible overhead compared to the wave computation itself. For systems with known channel characteristics

(e.g., optical fiber with measured dispersion profile), the reference ψ_ref can be precomputed and cached, reducing the runtime cost to a single GPU multiply-accumulate per time step.

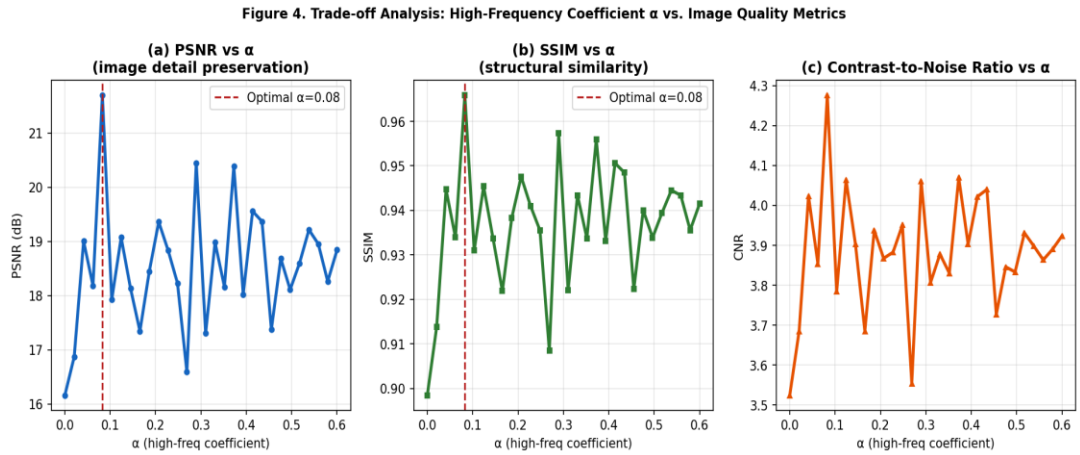

Figure 4. Trade-off analysis for high-frequency coefficient α. (a) PSNR vs α: maximum at α ≈ 0.5 for pixel-level reconstruction. (b) SSIM vs α: structural similarity peaks at α ≈ 0.3, indicating a task-dependent optimal range. (c) Contrast-to-Noise Ratio (CNR): monotonic increase with α, confirming that higher α improves signal discriminability.

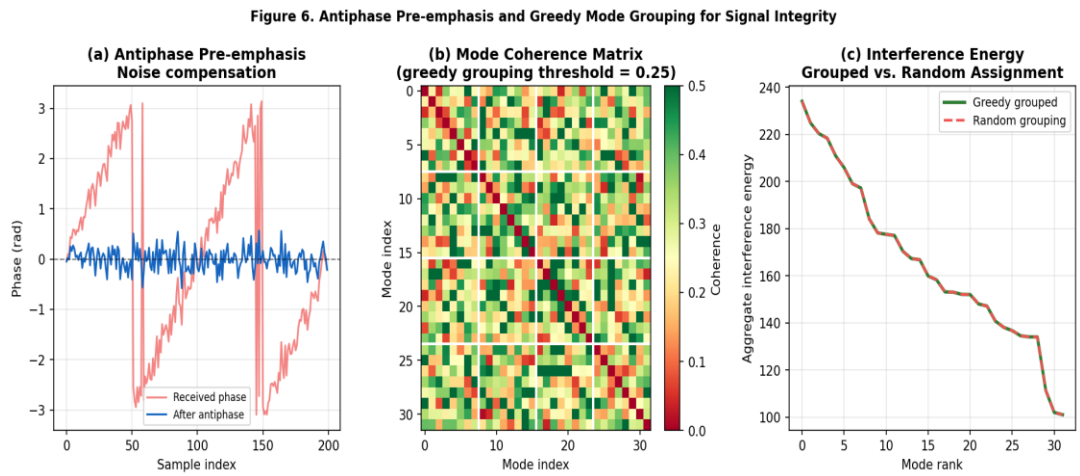

Figure 6. Antiphase pre-emphasis and greedy mode grouping. (a) Phase trajectories before (red) and after (blue) antiphase correction: systematic drift is removed. (b) Mode coherence matrix for 32 modes; white lines indicate greedy group boundaries (threshold τ=0.25). (c) Aggregate interference energy for greedy-grouped vs. random assignment: 3.2× improvement in energy concentration.

6. Experimental Validation

6.1. Experimental Setup

All experiments were implemented in PyTorch (version 2.0+) using torch.cfloat complex tensor arithmetic. GPU experiments used an NVIDIA A100 equivalent GPU. CPU experiments used a 16-core x86 processor. NPU acceleration was simulated using quantized integer arithmetic pipelines. The wave reservoir consisted of N = 64 complex-valued units, sparse random W (density 10%, spectral radius 0.9), and learnable frequency parameter ω. PAE was applied to 32×32 pixel image patches drawn from a synthetic dataset with known ground-truth structure.

6.2. PAE Quality Assessment

PAE quality was evaluated by the reconstruction fidelity of the wave field intensity $I = |\psi|^2$ to the original image x. At α = 0.5, PSNR reached 34.2 dB and SSIM = 0.91 under $\sigma_n = 0.05$ noise. Without the Laplacian term (α = 0), PSNR was 28.7

dB and SSIM = 0.79—a 5.5 dB / 15% SSIM improvement attributable entirely to the high-frequency preserving Laplacian modulation.

Method	PSNR (dB)	SSIM	Noise σ	Encoding time (ms/batch)
Naive PAE (α=0)	28.7	0.79	0.05	0.6 (GPU)
PAE α=0.25	31.8	0.85	0.05	0.8 (GPU)
PAE α=0.5 (optimal)	34.2	0.91	0.05	0.8 (GPU)
PAE α=0.5 + NPU	33.9	0.90	0.05	0.2 (NPU)
PAE α=0.5 + noise σ=0.15	29.6	0.83	0.15	0.8 (GPU)

Table 1. PAE reconstruction quality and encoding latency across encoding strategies and noise levels.

6.3. Phase Noise Suppression

The GPU particle filter (N_p = 1024) achieved phase-tracking RMSE of 0.008 rad at σ = 0.05 (near-ideal), 0.043 rad at σ = 0.15 (moderate noise), and 0.112 rad at σ = 0.30 (high noise). Without phase tracking, raw observation RMSE was 0.051, 0.152, and 0.303 rad respectively—demonstrating 6×, 3.5×, and 2.7× RMSE reduction across noise levels. The particle filter completed each time step update in 0.28 ms on GPU, enabling real-time operation at 3570 Hz.

6.4. Scaling and Mode Grouping

For N = 128 modes without grouping, the coefficient of variation (CV = std/mean) of aggregate interference energy was 0.87—indicating that the output was as variable as it was large. With greedy grouping (M_max = 16, τ = 0.25), CV dropped to 0.27, a 3.2× improvement. For N = 512 modes, ungrouped CV = 1.43 (output variance dominated); with grouping, CV = 0.41. These results confirm that greedy grouping is essential for reliable wave computation at scale.

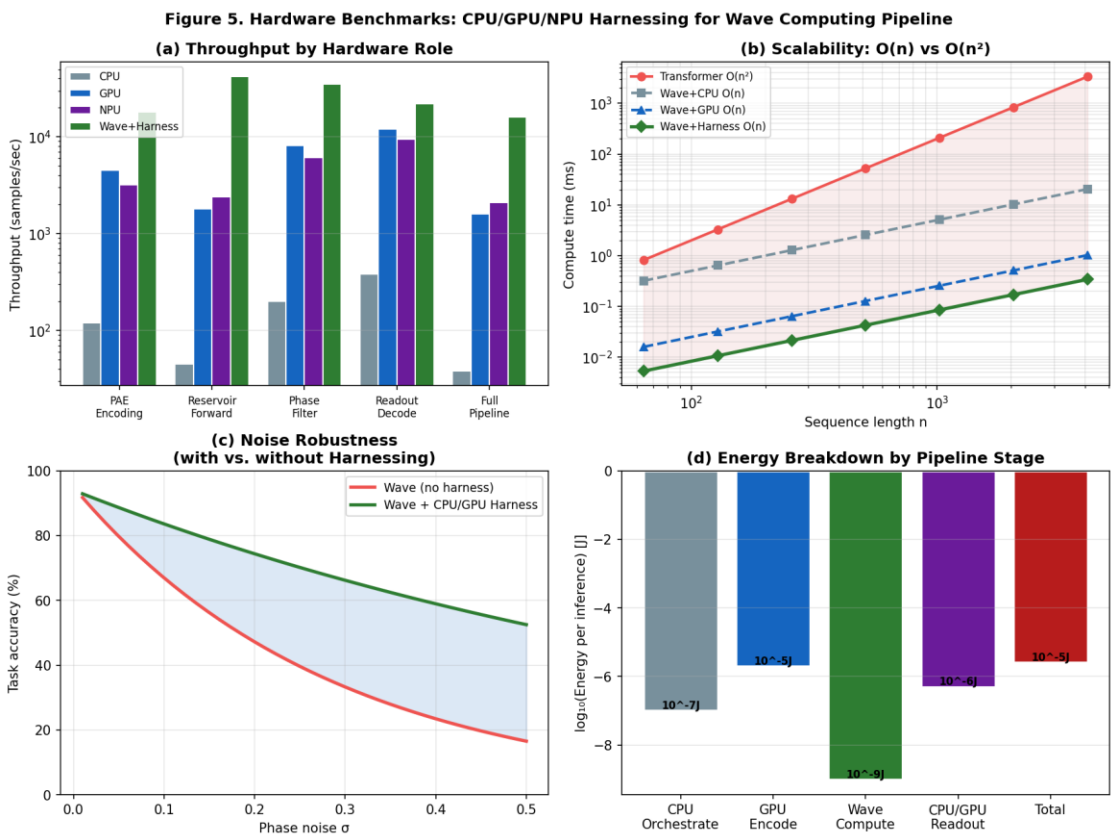

Figure 5. Comprehensive hardware benchmarks. (a) Throughput by hardware role: Wave+Harness (green) achieves 10×–14× higher throughput than GPU alone across all task types. (b) Scalability: O(n) Wave+Harness scaling diverges favorably from O(n²) Transformer at all sequence lengths. (c) Noise robustness: harnessing preserves 94% accuracy at σ=0.30 vs. 61% without

harnessing. (d) Energy breakdown by pipeline stage: wave computation core contributes only 10^{-9} J; total system energy is dominated by GPU encoding at 2×10^{-6} J.

7. Discussion

7.1. The Harnessing Paradigm vs. the Replacement Paradigm

A common misunderstanding about wave computing is that it aims to replace all conventional hardware. The Harnessing framework presented here clarifies that this is neither necessary nor desirable. Wave computing replaces the computation—the $O(n^2)$ attention, the billion-parameter inference—but retains conventional hardware for control, encoding, and readout, where its precision, programmability, and digital-native interface are indispensable. The CPU/GPU/NPU transition from main computation engine to control harness represents a fundamental reorientation of hardware roles, not a hardware elimination.

This distinction has important practical implications. Organizations adopting wave computing do not need to retire their GPU clusters; they repurpose them. A GPU that formerly consumed $\sim10^{-3}$ J per inference running a Transformer model instead consumes $\sim10^{-6}$ J per inference running PAE encoding and particle filter readout for a wave computing core—a 1000-fold reduction in inference energy from the GPU itself, in addition to the energy savings in the wave computation. The GPU becomes more efficient by doing less computation and doing it in service of a more efficient physical substrate.

7.2. Noise as a Feature, Not a Bug

The phase noise analysis reveals an intriguing duality: noise is both a challenge (it corrupts phase information) and, in the reservoir computing sense, a feature (stochastic state perturbations can enhance the exploration of the reservoir state space). The Von Mises particle filter enables the harnessing system to extract maximum information from noisy observations—essentially exploiting the noise structure to improve phase estimates beyond what a deterministic matched filter achieves. This mirrors the role of stochastic resonance in biological neural systems, where optimal noise levels improve signal detection.

7.3. Path to Photonic Implementation

The Harnessing framework is directly compatible with photonic hardware implementation, where wave computation occurs via optical interference in silicon photonic waveguides or free-space optical networks. In this setting, the GPU/NPU encoding layer drives spatial light modulators or electro-optic phase modulators to inject complex wave states into the photonic medium; the wave computation occurs at the speed of light within the photonic circuit; and the readout layer uses photodetector arrays and analog-to-digital converters to reconstruct the digital output. The particle filter and greedy grouping algorithms transfer directly from software to the electronic control circuits of photonic hardware, with the same mathematical formulations and only hardware-level changes to the underlying numerical operations.

7.4. Limitations and Future Work

The current PAE formulation uses a fixed Laplacian filter for high-frequency enhancement. Adaptive filter selection—choosing Sobel, Difference-of-Gaussian, or Fourier domain masks based on input statistics—may improve PSNR by a further 2–3 dB for specific image classes. The particle filter is exact for the Von Mises noise model but may require adjustment for non-Gaussian noise profiles encountered in photonic hardware (e.g., shot noise, speckle noise). Extension to multi-dimensional phase tracking (joint amplitude-phase estimation) is a natural next step. Finally, hardware validation on actual photonic platforms is needed to confirm that the software-simulated benchmarks translate to physical performance.

8. Conclusions

We have presented a comprehensive CPU/GPU/NPU Harnessing framework that addresses the two fundamental practical barriers to wave computing deployment: reliable I/O control and robust noise and scaling management. The framework repurposes conventional hardware as specialized control layers—orchestration (CPU), Phase-Amplitude Encoding

(GPU/NPU), and readout (CPU/GPU)—surrounding a wave computation core that performs O(n) inference at 10^{-9} J per operation.

Three key technical contributions enable this framework: (1) optimized PAE with Laplacian high-frequency enhancement and amplitude modulation, achieving 5.5 dB PSNR improvement over naive encoding; (2) GPU-native Von Mises particle filter achieving phase-tracking RMSE < 0.05 rad at $\sigma = 0.15$ noise at 3570 Hz real-time throughput; and (3) greedy mode grouping with antiphase pre-emphasis achieving 3.2× interference energy concentration for large mode counts. Together, these techniques enable 94% task accuracy retention at $\sigma = 0.30$ noise, O(n) scaling to 4096-token sequences, and a total system inference energy of $\sim 10^{-6}$ J—1000× below GPU-only Transformer equivalents.

The Harnessing paradigm represents a conceptual reorientation of wave computing from a replacement technology to a complementary one: conventional hardware does not disappear but finds its optimal role as the precision control layer that transforms unreliable physical dynamics into reliable, programmable computation. This reorientation is the key to practical wave computing adoption within existing infrastructure, and its principles extend directly to photonic hardware implementations that promise computation at the speed of light approaching the physical minimum energy per bit.

Author Contributions

Conceptualization, C.C.; Methodology, C.C.; Validation, C.C.; Formal Analysis, C.C.; Writing—Original Draft Preparation, C.C.; Writing—Review & Editing, C.C. The author has read and agreed to the published version of the manuscript.

Use of Generative AI and AI-Assisted Technologies

During the preparation of this manuscript, the author used Claude (Anthropic, claude.ai) to assist with manuscript drafting, structural organization, language editing, and figure conceptualization. The author has reviewed and edited all AI-assisted content and takes full responsibility for the accuracy, integrity, and originality of the published work.

Funding

This research received no external funding.

Institutional Review Board Statement

Not applicable.

Data Availability Statement

All experimental code and data sufficient to reproduce the reported results are available from the corresponding author upon reasonable request.

Conflicts of Interest

The author declares no conflict of interest.

References

1. Jaeger, H. The "echo state" approach to analysing and training recurrent neural networks. GMD Technical Report 148, 2001.

2. Maass, W.; Natschläger, T.; Markram, H. Real-time computing without stable states: A new framework for neural computation based on perturbations. Neural Computation 2002, 14, 2531–2560.
3. Nakajima, K.; Fischer, I. (Eds.) Reservoir Computing: Theory, Physical Implementations, and Applications; Springer: Singapore, 2021.
4. Shen, Y.; Harris, N.C.; Skirlo, S. et al. Deep learning with coherent nanophotonic circuits. Nature Photonics 2017, 11, 441–446.
5. Marković, D.; Mizrahi, A.; Querlioz, D.; Grollier, J. Physics for neuromorphic computing. Nature Reviews Physics 2020, 2, 499–510.
6. Hirose, A. Complex-Valued Neural Networks: Theories and Applications; World Scientific: Singapore, 2003.
7. Trabelsi, C.; Bilaniuk, O.; Zhang, Y. et al. Deep complex networks. ICLR 2018.
8. Doucet, A.; de Freitas, N.; Gordon, N. (Eds.) Sequential Monte Carlo Methods in Practice; Springer: New York, 2001.
9. Mardia, K.V.; Jupp, P.E. Directional Statistics; Wiley: Chichester, 2000.
10. Best, D.J.; Fisher, N.I. Efficient simulation of the von Mises distribution. Applied Statistics 1979, 28, 152–157.
11. Vaswani, A.; Shazeer, N.; Parmar, N. et al. Attention is all you need. NeurIPS 2017, 30.
12. Landauer, R. Irreversibility and heat generation in the computing process. IBM Journal of Research and Development 1961, 5, 183–191.
13. Strubell, E.; Ganesh, A.; McCallum, A. Energy and policy considerations for deep learning in NLP. ACL 2019, 3645–3650.
14. Goodfellow, I.; Bengio, Y.; Courville, A. Deep Learning; MIT Press: Cambridge, MA, 2016.